
Software Requirements Specification

**for
an OCR application**

Submitted revision

**Prepared by the Software Engineering team
consisting of M.W., D.F. and K.M.**

**Jagiellonian University in Krakow
compulsory student project**

16th June, 2025

Table of Contents

Table of Contents	ii
Revision History	Error! Bookmark not defined.
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	1
1.5 References.....	1
2. Overall Description	2
2.1 Product Perspective.....	2
2.2 Product Functions	2
2.3 User Classes and Characteristics	2
2.4 Operating Environment.....	2
2.5 Design and Implementation Constraints	3
2.6 User Documentation	4
2.7 Assumptions and Dependencies	4
3. External Interface Requirements	4
3.1 User Interfaces	4
3.2 Hardware Interfaces	5
3.3 Software Interfaces	5
3.4 Communications Interfaces	5
4. System Features	6
4.1 Image upload and processing.....	Error! Bookmark not defined.
5. Other Nonfunctional Requirements.....	7
5.1 Performance Requirements	7
5.2 Security Requirements	7

1. Introduction

1.1 Purpose

The purpose of this document is to outline the software requirements for the product that is an OCR application developed with Django and React. The product was developed by the team mentioned on the cover. This document covers the entire system in its first release. Due to the nature of the project, we have refrained from a numbered release system, however, for completeness, the current product may be referred to as version 1.0.

1.2 Document Conventions

This document does not contain any special standards or typographical conventions.

1.3 Intended Audience and Reading Suggestions

The intended audience of this document is the instructor of the laboratory section of the Software Engineering course at JU, as well as any parties that may be interested in its viewing (e.g. through visiting the project's version control site on GitHub). The rest of the document contains a more detailed outline of the project, as opposed to the "project scope and vision" document. The document attempts to be detailed in that it is precise; its brevity allows any reader to become familiar with it in a short time span.

1.4 Product Scope

The product consists of a frontend built with React that acts as a UI for a Django backend. The Django backend contains two OCR (Optical Character Recognition) models – PaddleOCR and EasyOCR. The language being detected in OCR can be changed, PaddleOCR is used for English language tasks whereas EasyOCR is used for their Polish counterparts. The product aims to provide an open and free alternative to commercial OCR. We do not assess that this is an utmost realistic goal, but we hope to provide a decent proof-of-concept.

1.5 References

This document frequently mentions tools and solutions taken from:

- The Django framework for Python (<https://www.djangoproject.com/>)
- The React framework for frontend development (<https://react.dev/>)

2. Overall Description

2.1 Product Perspective

This OCR application is a new, self-contained product developed as a proof-of-concept for academic purposes. It is not a replacement for existing systems but rather an alternative approach to commercial OCR solutions. The system consists of three main components: a React frontend for user interaction, a Django REST API backend for business logic, and integrated OCR models (PaddleOCR and EasyOCR) for text recognition. The application operates as a web-based solution requiring no additional software installation from the user perspective.

2.2 Product Functions

The major functions the product must perform include:

- **Image Upload and Processing:** Accept .jpg and .png image files and automatically adjust resolution
- **Text Recognition:** Extract text from uploaded images using OCR models
- **Language Selection:** Switch between English (PaddleOCR) and Polish (EasyOCR) language recognition
- **Confidence Display:** Show recognition confidence levels for extracted words
- **Text Editing:** Allow users to modify recognized text with paragraph width adjustment
- **Export Functionality:** Export processed text to multiple editable formats (docx, txt)
- **User Authentication:** Provide user registration and login via Google OAuth
- **File Management:** Allow registered users to access their previously processed files
- **Error Handling:** Display informative error messages for various input problems
- **Contact Support:** Provide contact form for user communication with administrators

2.3 User Classes and Characteristics

Primary Users - Students (High Priority)

- **Characteristics:** Tech-savvy, frequent note-takers, familiar with web applications
- **Usage Pattern:** Regular use for digitizing handwritten notes
- **Technical Expertise:** Moderate to high
- **Primary Functions:** Upload images, process text, export results

Secondary Users - Advanced Users (Medium Priority)

- Characteristics: Technically sophisticated, require detailed system feedback
- Usage Pattern: Occasional use with focus on quality metrics
- Technical Expertise: High
- Primary Functions: All basic functions plus confidence level analysis

Polish Language Users (Medium Priority)

- Characteristics: Polish speakers requiring diacritic character support
- Usage Pattern: Regular use for Polish text recognition
- Technical Expertise: Moderate
- Primary Functions: Upload images, enable Polish language mode, process text

System Administrators (Low Priority)

- Characteristics: Technical staff responsible for system maintenance
- Usage Pattern: Monitoring, troubleshooting, user support
- Technical Expertise: High
- Primary Functions: System monitoring, user support via contact forms

2.4 Operating Environment

Client-Side Requirements:

- Web browser supporting modern JavaScript (tested for Firefox 139+ and Edge 136+)
- Internet connection for accessing the web application
- Operating System: Platform independent (tested for Windows and Linux)

Server-Side Environment:

- Python 3.13+ with Django framework
- Node.js environment for React frontend
- Compatible with standard web hosting environments
- Database: Google Firebase

2.5 Design and Implementation Constraints

Technology Constraints:

- Must use Django REST Framework for backend API development
- Frontend must be implemented using React framework
- OCR processing limited to PaddleOCR and EasyOCR models
- File format support restricted to .jpg and .png images
- Authentication must use Google OAuth for simplicity

Performance Constraints:

- Processing time must not exceed 30 seconds per file
- Image size limitations to prevent server overload

Security Constraints:

- No storage of sensitive user data beyond authentication tokens
- Clear warnings about not uploading confidential documents
- Secure file handling and temporary storage practices

2.6 User Documentation

The following user documentation will be provided:

- **Contact Information:** Administrator contact form integrated into the application

2.7 Assumptions and Dependencies

Assumptions:

- Users have stable internet connections for uploading images
- Target images contain readable text suitable for OCR processing
- Users understand basic web application navigation
- Google OAuth service remains available and accessible

Dependencies:

- PaddleOCR library availability and compatibility
- EasyOCR library availability and compatibility
- Google OAuth API accessibility
- Django and React framework stability
- Web hosting environment supporting Python and Node.js applications

3. External Interface Requirements

3.1 User Interfaces

General UI Characteristics:

- Responsive design adapting to various screen sizes and devices
- Consistent visual theme with dark text colors and light/white/gray backgrounds
- Rounded corners on rectangular elements and smooth text rendering (antialiasing)
- Intuitive navigation suitable for student users

Main Interface Components:

- **Upload Interface:** Drag-and-drop file upload area with file format indicators
- **Processing Interface:** Progress indicators and real-time status updates
- **Results Interface:** Text display area with editing capabilities and confidence indicators
- **Export Interface:** Format selection and download functionality

- **Authentication Interface:** Google OAuth login integration
- **Settings Interface:** Language selection and paragraph width adjustment controls
- **Contact Interface:** Support form with administrator communication

Error Display Standards:

- Non-intrusive error notifications that don't disrupt user workflow
- Specific error categorization (file format, image quality, processing errors)
- Clear instructions for error resolution
- Warning messages for data sensitivity

3.2 Hardware Interfaces

The application operates entirely through standard web browser interfaces and requires no direct hardware interface implementations (except for development). All hardware interactions are mediated through the user's web browser and operating system.

3.3 Software Interfaces

Django Backend Interfaces:

- **OCR Model Integration:**
 - PaddleOCR library for English text recognition
 - EasyOCR library for Polish text recognition
 - Input: Image file data, language selection
 - Output: Extracted text, confidence scores
- **Database Interface:**
 - Google Firebase
 - User authentication data storage
 - File processing history for registered users
- **Authentication Interface:**
 - Google OAuth 2.0 API integration
 - User session management
 - Secure token handling

React Frontend Interfaces:

- **REST API Communication:**
 - HTTP requests to Django backend
 - JSON data format for all API communications
 - File upload via multipart form data
 - Authentication token inclusion in requests

3.4 Communications Interfaces

HTTP/HTTPS Protocol:

- RESTful API design following standard HTTP methods
- HTTPS encryption for all data transmission
- JSON format for structured data exchange
- Multipart form data for file uploads

Authentication Communications:

- OAuth 2.0 protocol implementation
- Secure token exchange with Google services
- Session management via HTTP cookies

Error Communication:

- Standardized HTTP status codes
- Structured error response format in JSON
- Client-side error handling and user notification

4. System Features

4.1 Image upload and processing

4.1.1 Description and Priority

Priority: High Core functionality allowing users to upload image files (.jpg, .png) for OCR processing. The system must automatically validate file formats, adjust image resolution, and prepare images for text recognition processing.

4.1.2 Stimulus/Response Sequences

- User selects image file via upload interface
- System validates file format and size
- System displays upload progress
- System processes image (resolution adjustment, preprocessing)
- System confirms successful upload and readiness for OCR processing

4.1.3 Functional Requirements

REQ-1: The system shall accept image files in .jpg and .png formats only.

REQ-2: The system shall provide an offline proof-of-concept user interface.

REQ-3: The system shall provide export capabilities for both .docx and .txt formats.

REQ-4: The system shall provide font size (.docx) and paragraph width (.txt) modifiers.

REQ-5: The system shall provide a modifier to be able to view word prediction confidence.

REQ-6: The system shall automatically process and modify the resolution of inputs images.

REQ-7: The system shall provide Polish handwritten text recognition.

REQ-8: The system shall handle errors internally and not fail when encountering one.

REQ-9: The system shall inform the user about not sending sensitive data.

REQ-10: The system shall inform the user whether the input image is too dark or too bright.

5. Other Nonfunctional Requirements

5.1 General Requirements

Processing Time: All OCR operations must complete within 30 seconds to ensure acceptable user experience and prevent timeout issues.

Dynamic interface: The web UI should scale properly and adjust to window size.

UI consistency: The UI should have a uniform color scheme, an anti-aliased font and round boxes.

5.2 Security Requirements

Authentication Security: Google OAuth integration must follow OAuth 2.0 security best practices including secure token handling.